

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное
учреждение высшего образования
ИРКУТСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

Институт информационных технологий и анализа
данных
Центр программной инженерии

ОТЧЕТ

к лабораторной работе №2 по дисциплине:

«Методы оптимизации»

Решение задачи условной оптимизации

наименование темы

Выполнил студент

группы: ИСИБ-23-1

шифр группы

подпись

Третьяк А. Н.

Фамилия ИО

Проверил:

шифр группы

подпись

Серышева И. А.

Фамилия ИО

Иркутск 2025 г

Вариант 20

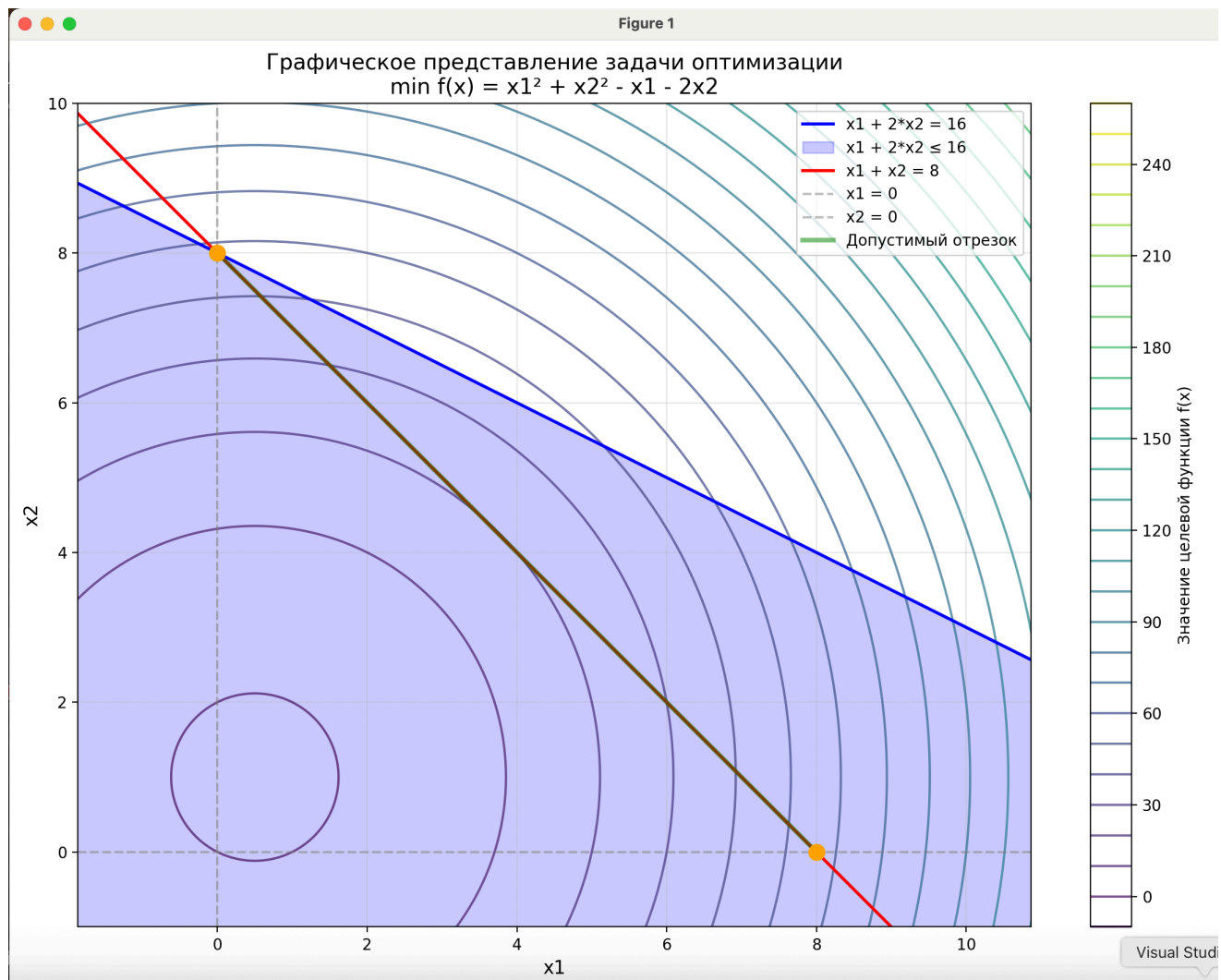
1. Цель работы: знакомство с методами штрафных функций (метода внутренней точки и метода внешней точки) для решения задач условной оптимизации.

2. Постановка задачи

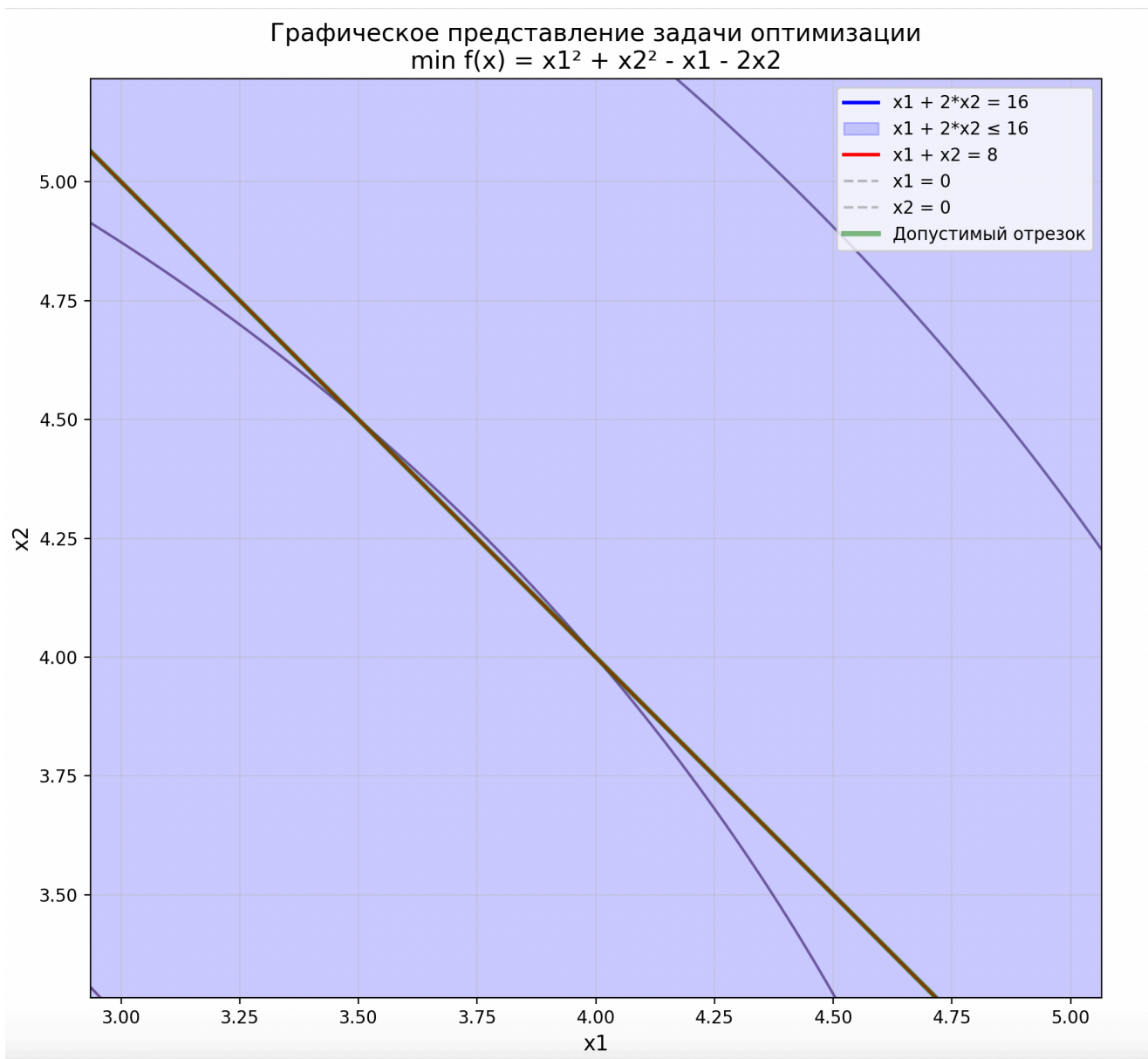
20	$\begin{aligned} \min f &= x_1^2 + x_2^2 - x_1 - 2x_2 \\ x_1 + 2x_2 &\leq 16 \\ x_1 + x_2 &= 8 \\ x_1 &\geq 0, \\ x_2 &\geq 0 \end{aligned}$	Комбини- рованный метод	Метод Ньютона
----	--	-------------------------------	------------------

3. Ход работы

3.1 Графическая интерпретация



По полученному изображению сразу можно сказать, что минимум функции будет примерно равен 20, так как линия равного уровня функции со значением 20 примерно попадает на допустимый отрезок.



В приближении мы можем увидеть, что минимум достигается в $(x_1 = 3,75; x_2 = 4,25)$. Если подставим в исходную функцию, то получим значение 19,875. Найдём математически:

Первым делом подметим очевидное: для поиска решения можем не учитывать неравенство, так как равенство полностью находится внутри этой области при положительных x_1, x_2 (мы не выйдем за положительность, ведь очевидно, что решение находится в положительных координатах). Нам остается только выразить одну переменную из равенства через другую, подставить в функцию, которую оптимизируем и найти производную. Приравнявая ее к нулю мы получим одну координату, через равенство найдем другую и это и будет оптимальным решением.

$$x_2 = 8 - x_1$$

$$f = x_1^2 + x_2^2 - x_1 - 2x_2 = x_1^2 + (8 - x_1)^2 - x_1 - 2(8 - x_1)$$

$$f = 2x_1^2 - 15x_1 + 48$$

$$f' = 4x_1 - 15 = 0$$

$$x_1 = 3,75$$

$$x_2 = 4,25$$

$$f = 19,875$$

3.2 Построение штрафной функции

Мы будем использовать комбинированный метод, а значит штрафная функция будет иметь вид:

$$P(X, r) = f(X) + \frac{1}{\sqrt{r}} \sum_{j=m+1}^p h_j^2(X) + r \sum_{i=1}^m \frac{1}{g_i(X)}$$

$$P(X, r) = x_1^2 + x_2^2 - x_1 - 2x_2 + \frac{1}{\sqrt{r}}(x_1 + x_2 - 8)^2 +$$

$$r\left(\frac{1}{-x_1 - 2x_2 + 16} + \frac{1}{x_1} + \frac{1}{x_2}\right)$$

3.3 Градиент и матрица Гессе функции P(X, r)

$$\frac{\partial P}{\partial x_1} = 2x_1 - 1 + \frac{2}{\sqrt{r}}(x_1 + x_2 - 8) + r\left(\frac{1}{(-x_1 - 2x_2 + 16)^2} - \frac{1}{x_1^2}\right)$$

$$\frac{\partial P}{\partial x_2} = 2x_2 - 2 + \frac{2}{\sqrt{r}}(x_1 + x_2 - 8) + r\left(\frac{1}{(-x_1 - 2x_2 + 16)^2} - \frac{1}{x_2^2}\right)$$

$$\frac{\partial^2 P}{\partial x_1^2} = 2 + \frac{2}{\sqrt{r}} + r\left(\frac{2}{(-x_1 - 2x_2 + 16)^3} + \frac{2}{x_1^3}\right)$$

$$\frac{\partial^2 P}{\partial x_2^2} = 2 + \frac{2}{\sqrt{r}} + r\left(\frac{8}{(-x_1 - 2x_2 + 16)^3} + \frac{2}{x_2^3}\right)$$

$$\frac{\partial^2 P}{\partial x_1 \partial x_2} = \frac{2}{\sqrt{r}} + \frac{4r}{(-x_1 - 2x_2 + 16)^3}$$

3.4 Вычислительная процедура методом Ньютона

1 шаг: выбираем такую точку, которая будет удовлетворять неравенства. Например $X^0 = (2; 3)$. $2 + 6 = 8 < 16$. Начальное значение $r^0 = 10$.

2 шаг: вычисляем градиент и матрицу Гессе в выбранной точке:

$$\nabla P(2,3) = \begin{pmatrix} -1,2411 \\ 1,3040 \end{pmatrix}, \quad H(2,3) = \begin{pmatrix} 5,1715 & 0,7106 \\ 0,7106 & 3,5294 \end{pmatrix}$$

Гессиан положительно определен, все миноры положительные.

3 шаг: выбираем направление движения и величину шага. Для этого нужно найти обратную матрицу гессиана.

$$H^{-1}(2,3) = \begin{pmatrix} 0,1989 & -0,0400 \\ -0,0400 & 0,2914 \end{pmatrix}$$

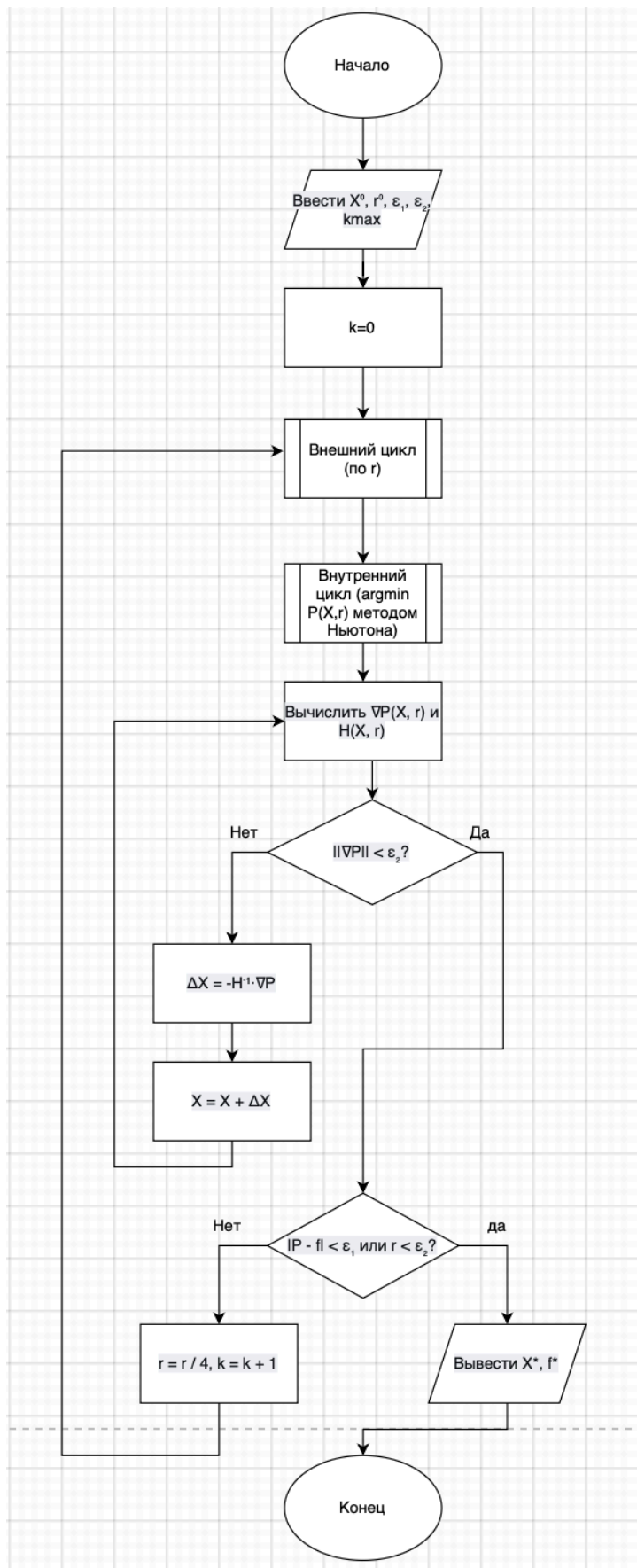
$$\Delta X = -H^{-1} \cdot \nabla P = \begin{pmatrix} 0,2990 \\ -0,4297 \end{pmatrix}$$

$$X_{\text{new}} = X_{\text{old}} + \Delta X = \begin{pmatrix} 2 \\ 3 \end{pmatrix} + \begin{pmatrix} 0,2990 \\ -0,4297 \end{pmatrix} = \begin{pmatrix} 2,2990 \\ 2,5703 \end{pmatrix}$$

Далее продолжаем двигаться, пока градиент и изменение значений координат не будут меньше малого значения (пусть равного 10^{-6})

4 шаг: меняем r и возвращаемся к шагу 2, критерием останова в данном случае будет $P - f$ меньше малого значения или r меньше малого значения (тоже например 10^{-6})

3.5 Блок-схема алгоритма



3.6 Решение задачи на ЭВМ

Начальная точка: (3, 4)
f = 14
h1 = x1 + x2 - 8 = -1
g1 = 16 - x1 - 2x2 = 5
P(X, r) = 22.14956109935017

r = 10
iter 0: X=(2.3342, 2.6308), ||grad||=6.64e+00
iter 10: X=(2.3287, 2.5855), ||grad||=3.66e-04
Сошлись за 14 итераций
X = (2.328682, 2.585443)
f = 4.607707
h1 = -3.09e+00
g1 = 8.50e+00
P(X, r) = 16.957520169216288

r = 2.5
iter 0: X=(2.4227, 2.8411), ||grad||=1.24e+00
Сошлись за 2 итераций
X = (2.422202, 2.843652)
f = 5.843915
h1 = -2.73e+00
g1 = 7.89e+00
P(X, r) = 12.799975756976867

r = 0.625
iter 0: X=(2.8429, 3.3269), ||grad||=4.57e+00
Сошлись за 2 итераций
X = (2.844492, 3.326715)
f = 9.660243
h1 = -1.83e+00
g1 = 6.50e+00
P(X, r) = 14.394439065339196

r = 0.15625
iter 0: X=(3.2162, 3.7120), ||grad||=6.50e+00
Сошлись за 2 итераций
X = (3.216383, 3.711782)
f = 13.482501
h1 = -1.07e+00
g1 = 5.36e+00
P(X, r) = 16.50866037787107

r = 0.0390625
iter 0: X=(3.3374, 3.8344), ||grad||=7.66e+00
iter 10: X=(3.4583, 3.9570), ||grad||=7.48e-03
Сошлись за 13 итераций
X = (3.458370, 3.957073)
f = 16.246235
h1 = -5.85e-01
g1 = 4.63e+00
P(X, r) = 18.00475710591581

r = 0.009765625
iter 0: X=(3.5278, 4.0269), ||grad||=8.37e+00
iter 10: X=(3.5972, 4.0968), ||grad||=8.17e-03
Сошлись за 11 итераций
X = (3.597154, 4.096792)
f = 17.932482

```

h1 = -3.06e-01
g1 = 4.21e+00
P(X, r) = 18.88776631918128

r = 0.00244140625
iter 0: X=(3.6344, 4.1342), ||grad||=8.76e+00
Сошлись за 10 итераций
X = (3.671691, 4.171594)
f = 18.868636
h1 = -1.57e-01
g1 = 3.99e+00
P(X, r) = 19.36754628176608

r = 0.0006103515625
iter 0: X=(3.6910, 4.1910), ||grad||=8.97e+00
Сошлись за 8 итераций
X = (3.710356, 4.210331)
f = 19.362612
h1 = -7.93e-02
g1 = 3.87e+00
P(X, r) = 19.6177016371653

r = 0.000152587890625
iter 0: X=(3.7202, 4.2202), ||grad||=9.08e+00
Сошлись за 7 итераций
X = (3.730053, 4.230047)
f = 19.616448
h1 = -3.99e-02
g1 = 3.81e+00
P(X, r) = 19.745443103997037

r = 3.814697265625e-05
iter 0: X=(3.7350, 4.2350), ||grad||=9.14e+00
Сошлись за 6 итераций
X = (3.739995, 4.239994)
f = 19.745127
h1 = -2.00e-02
g1 = 3.78e+00
P(X, r) = 19.809992714337778

r = 9.5367431640625e-06
iter 0: X=(3.7425, 4.2425), ||grad||=9.16e+00
Сошлись за 4 итераций
X = (3.744990, 4.244989)
f = 19.809913
h1 = -1.00e-02
g1 = 3.77e+00
P(X, r) = 19.84243890560113

r = 2.384185791015625e-06
iter 0: X=(3.7462, 4.2462), ||grad||=9.18e+00
Сошлись за 3 итераций
X = (3.747493, 4.247493)
f = 19.842419
h1 = -5.01e-03
g1 = 3.76e+00
P(X, r) = 19.858705059414575

r = 5.960464477539062e-07
iter 0: X=(3.7487, 4.2487), ||grad||=9.19e+00
Сошлись за 1 итераций

```

```
x = (3.748746, 4.248746)
f = 19.858700
h1 = -2.51e-03
g1 = 3.75e+00
P(X, r) = 19.866848927534818
```

Итог

```
x* = (3.748746, 4.248746)
```

```
f* = 19.858700
```

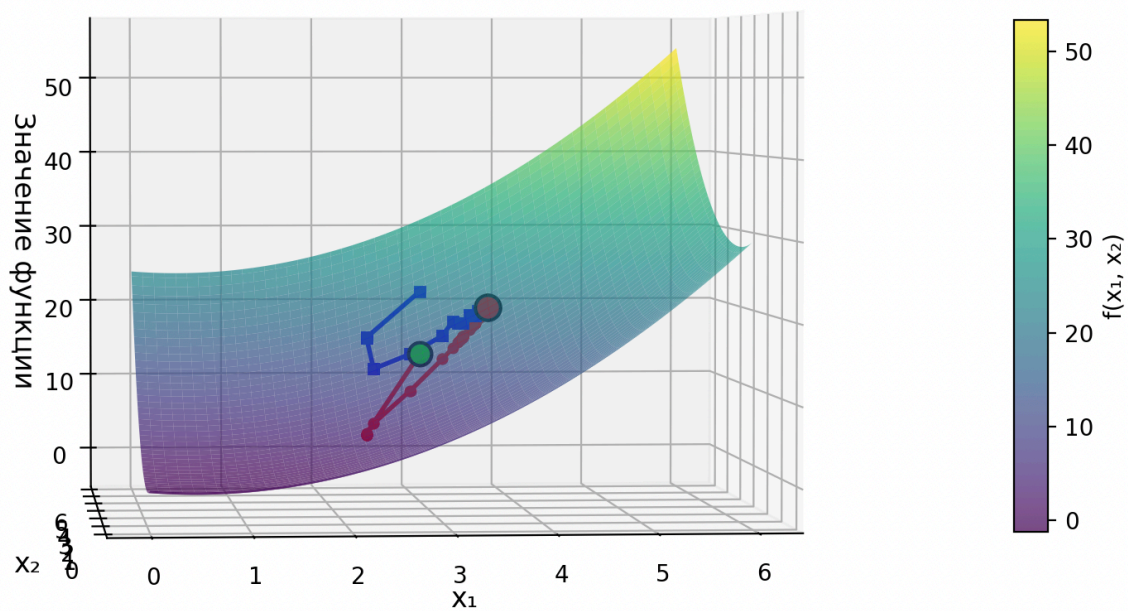
```
Проверка x1 + x2 - 8 = -2.51e-03
```

```
Теоретический оптимум: (3.75, 4.25), f = 19.875
```

```
96 кол-во итераций
```

Поверхность $f(x_1, x_2)$ с наложенными траекториями f и $P(X, r)$

- Траектория $f(X)$
- Траектория $P(X, r)$
- Начальная точка (f)
- Конечная точка (f^*)



3.7 Анализ траектории поиска для нескольких вариаций функции

Предыдущий эксперимент был запущен с поиском шага Armijo, с начальными условиями $x_1 = 3, x_2 = 4, r = 10$. Проверим, как будет себя вести метод при изменении начальной точки, стратегии выбора шага.

Первый эксперимент проведем с изначально выбранными точками за границей допустимых значений. Ожидаем, что $P(X, r)$ будет иметь большое значение как наказание за выход из области по неравенствам. Точки $x_1 = -3, x_2 = -4$

ИТОГ

$X^* = (3.748746, 4.248746)$

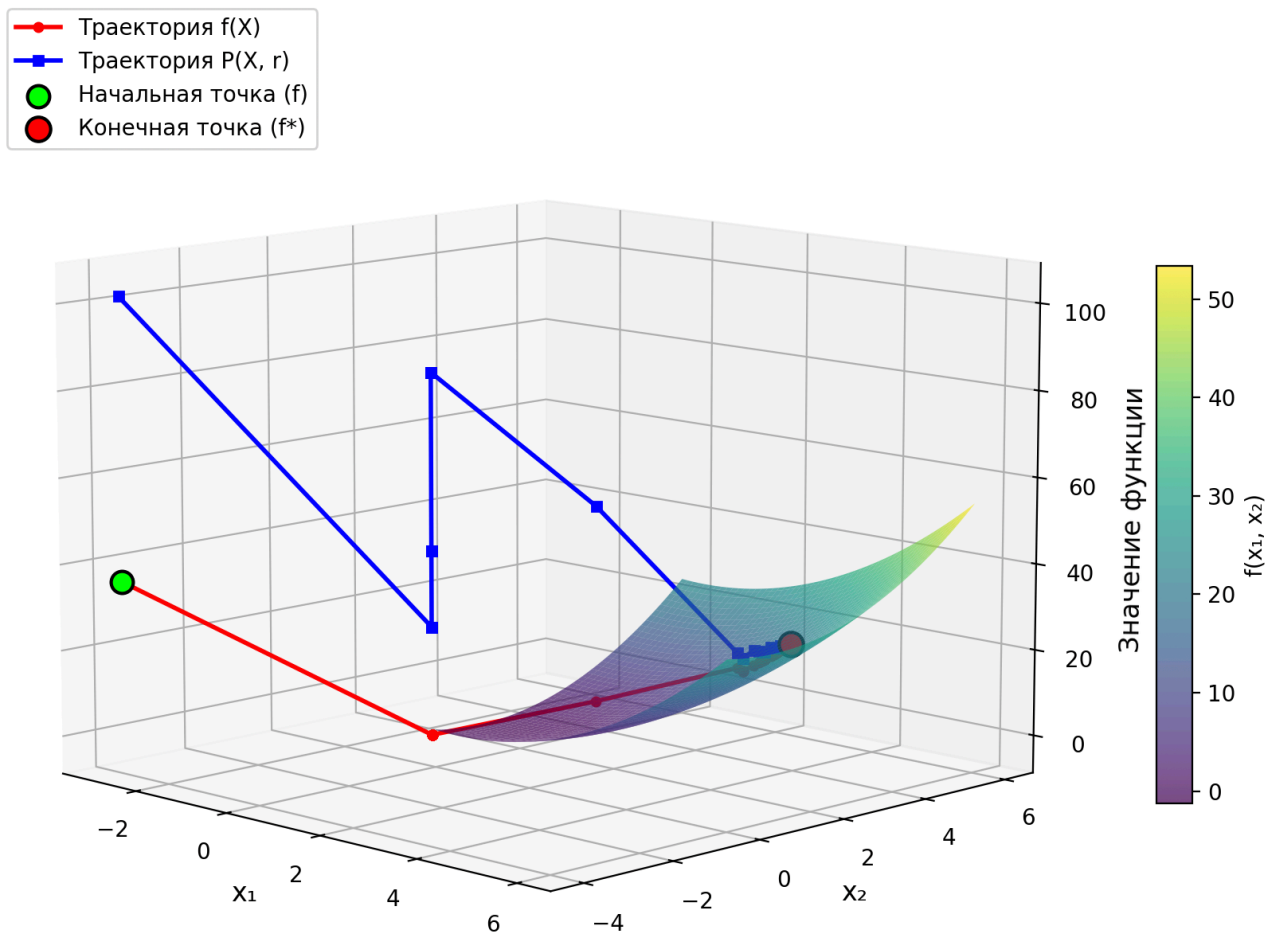
$f^* = 19.858700$

Проверка $x_1 + x_2 - 8 = -2.51e-03$

Теоретический оптимум: $(3.75, 4.25), f = 19.875$

234 кол-во итераций

Поверхность $f(x_1, x_2)$ с наложенными траекториями f и $P(X, r)$



Пришли к оптимуму медленнее, но все-таки пришли.

Теперь попробуем еще точку, принадлежащую равенству из ограничения. $x_1 = 5, x_2 = 3$

Итог

$X^* = (3.748746, 4.248746)$

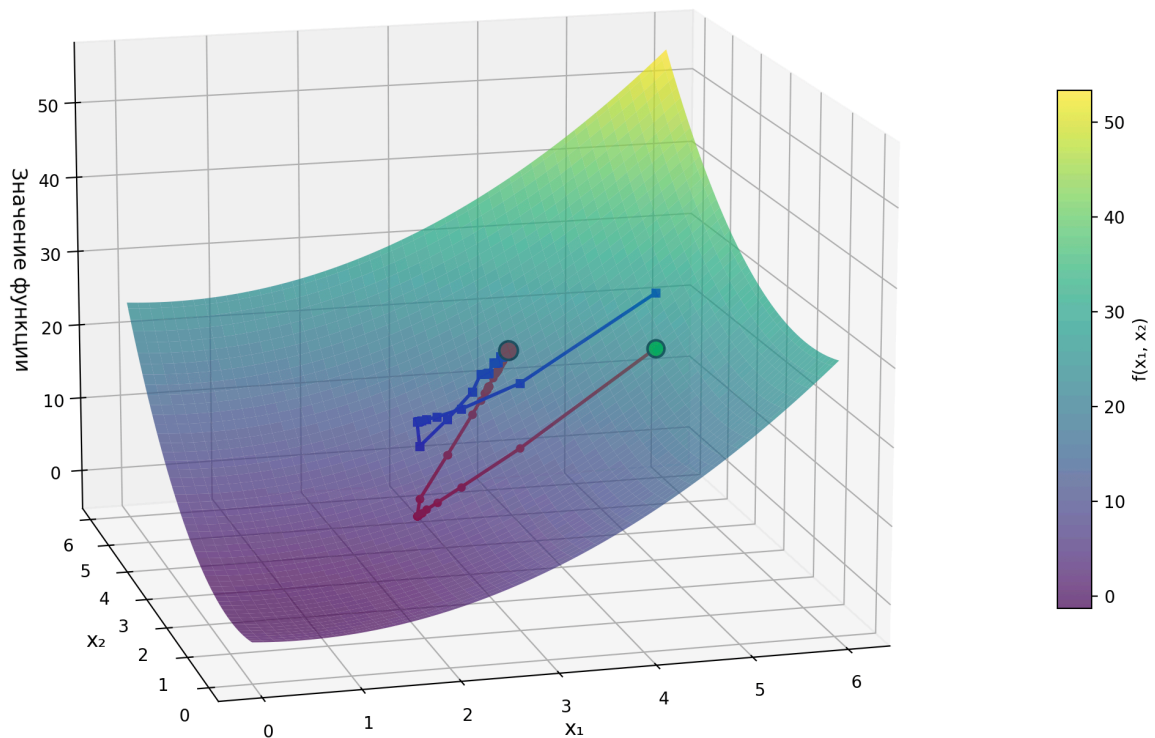
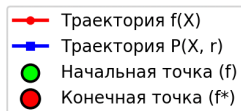
$f^* = 19.858700$

Проверка $x_1 + x_2 - 8 = -2.51e-03$

Теоретический оптимум: $(3.75, 4.25)$, $f = 19.875$

102 кол-во итераций

Поверхность $f(x_1, x_2)$ с наложенными траекториями f и $P(X, r)$



Сошлись быстрее, чем в прошлый раз. Пришли к решению. Из-за того, что из неравенств получаем штрафы.

Далее проведем эксперименты из тех же точек, но поиск используя стратегию подбора шага Вульфа.

Для $x_1 = 3, x_2 = 4$

Итог

$X^* = (3.748746, 4.248746)$

$f^* = 19.858700$

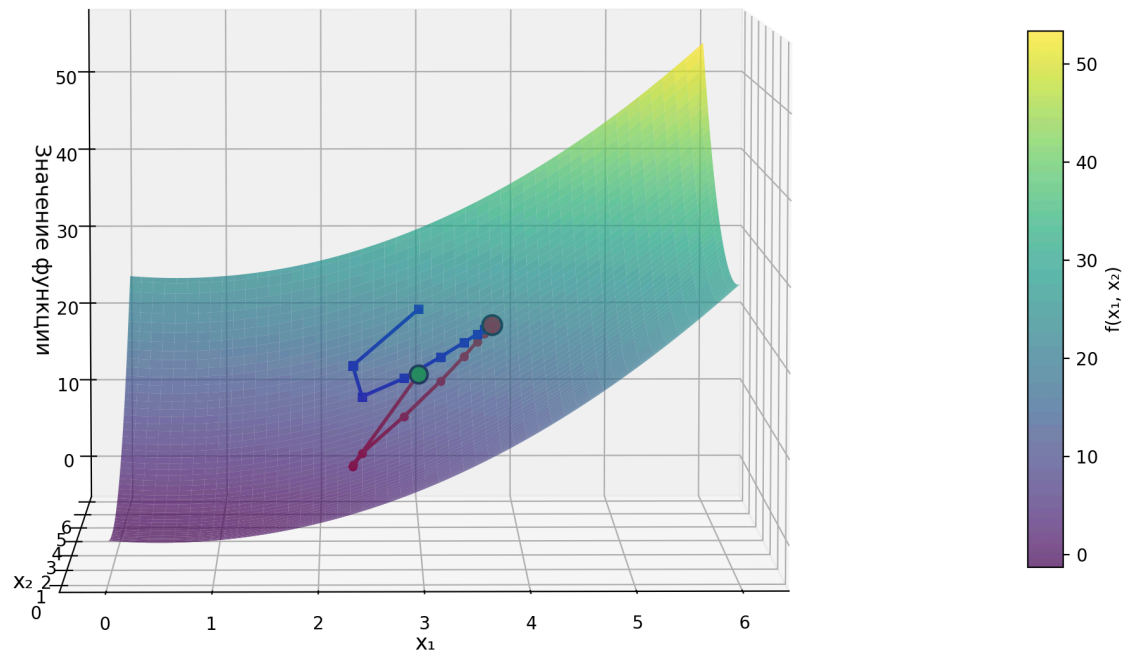
Проверка $x_1 + x_2 - 8 = -2.51e-03$

Теоретический оптимум: $(3.75, 4.25)$, $f = 19.875$

34 кол-во итераций

Поверхность $f(x_1, x_2)$ с наложенными траекториями f и $P(X, r)$

- Траектория $f(X)$
- Траектория $P(X, r)$
- Начальная точка (f)
- Конечная точка (f^*)



Для $x_1 = -3, x_2 = -4$

Итого

$X^* = (3.748746, 4.248746)$

$f^* = 19.858700$

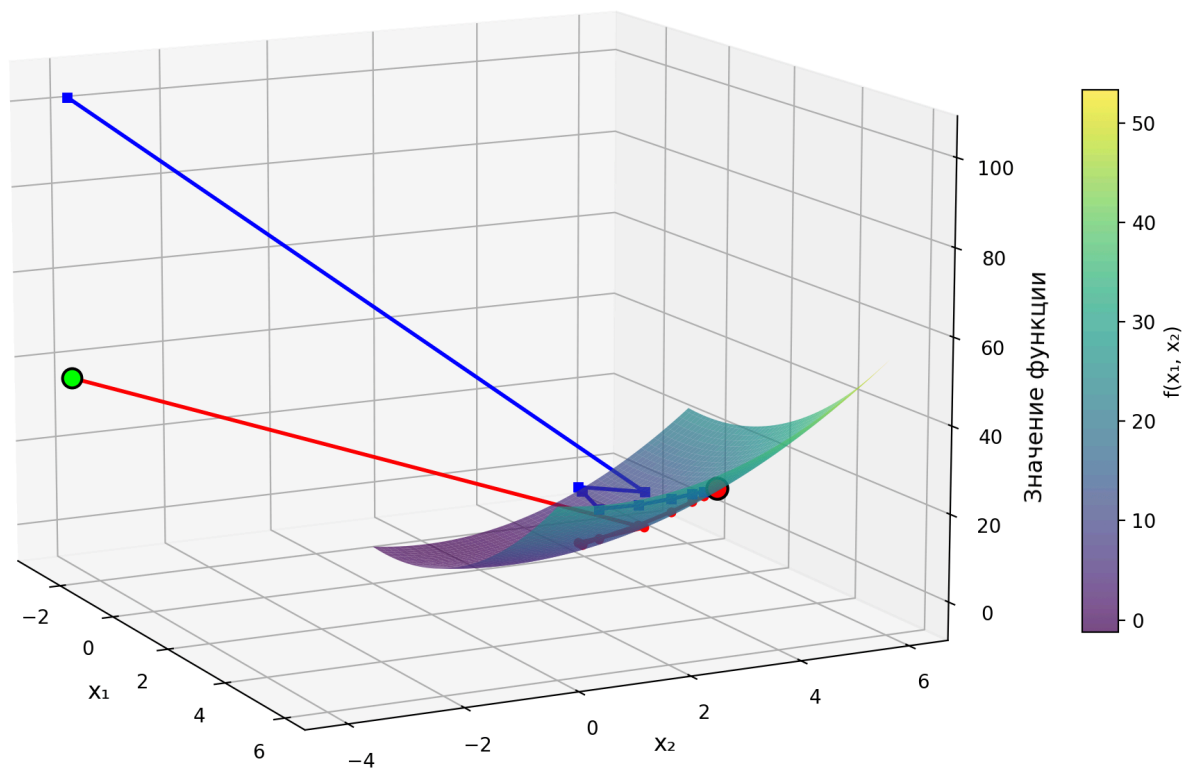
Проверка $x_1 + x_2 - 8 = -2.51e-03$

Теоретический оптимум: $(3.75, 4.25), f = 19.875$

36 кол-во итераций

Поверхность $f(x_1, x_2)$ с наложенными траекториями f и $P(X, r)$

- Траектория $f(X)$
- Траектория $P(X, r)$
- Начальная точка (f)
- Конечная точка (f^*)



Для $x_1 = 5, x_2 = 3$

Итог

$X^* = (3.748746, 4.248746)$

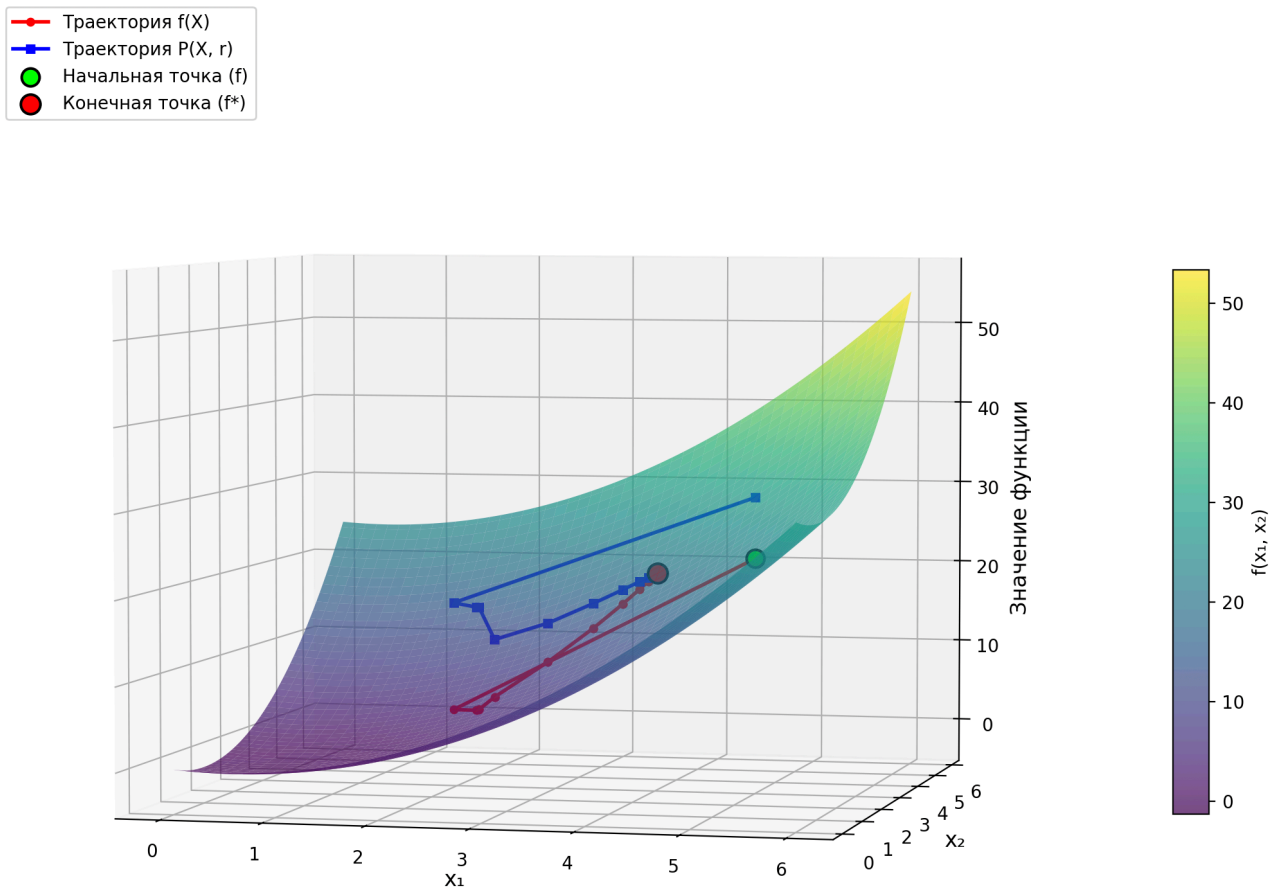
$f^* = 19.858700$

Проверка $x_1 + x_2 - 8 = -2.51e-03$

Теоретический оптимум: $(3.75, 4.25), f = 19.875$

35 кол-во итераций

Поверхность $f(x_1, x_2)$ с наложенными траекториями f и $P(X, r)$

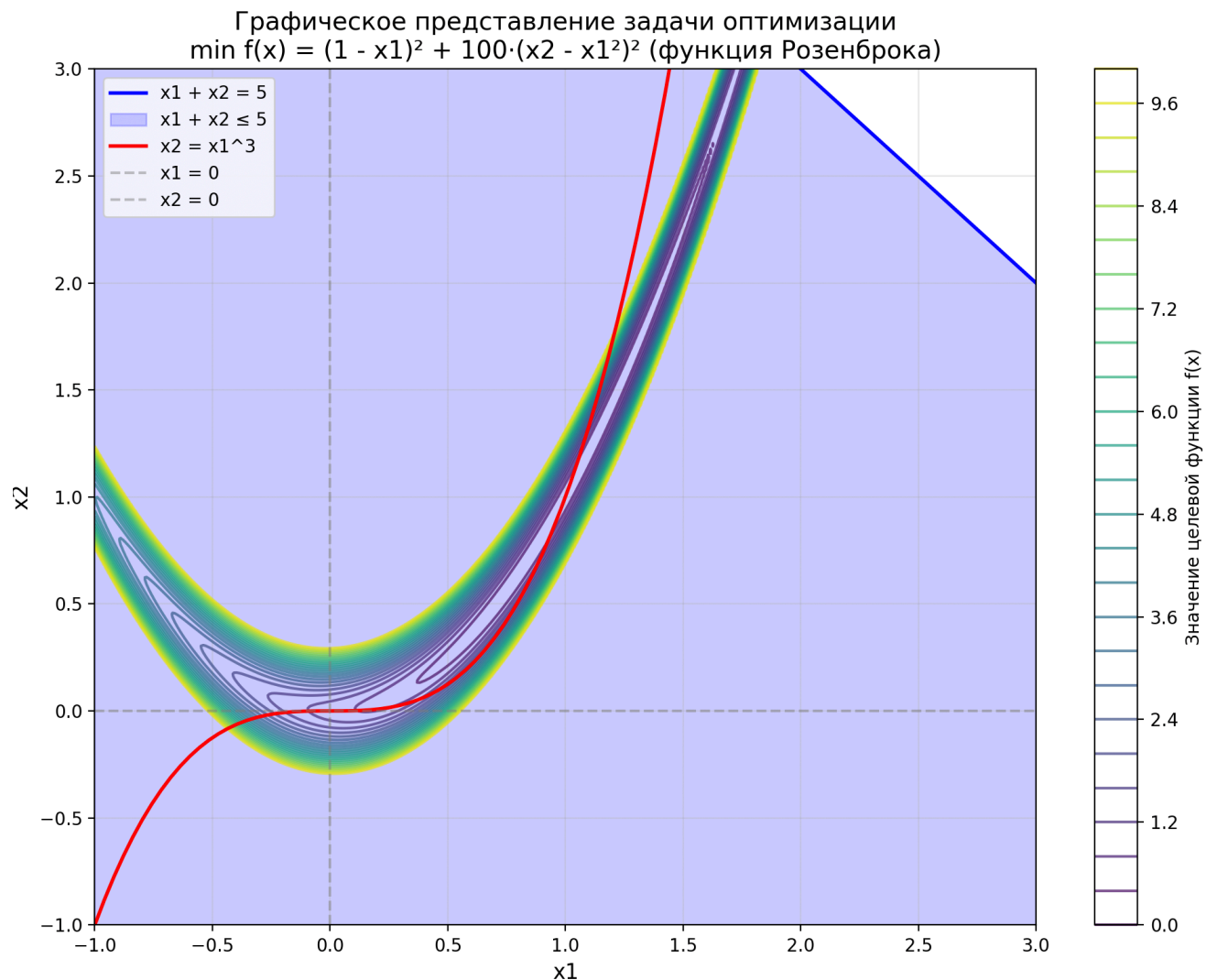


Сравнительная таблица:

№ эксперимента	Метод подбора шага	x_1		x_2		F		P		Кол-во итераций
1	Armijo	3	3.748	4	4.248	14	19.858	22.149	19.866	96
2	Armijo	-3	3.748	-4	4.248	36	19.858	101.688	19.866	234
3	Armijo	5	3.748	3	4.248	23	19.858	30.333	19.866	102
4	Wolfe	3	3.748	4	4.248	14	19.858	22.149	19.866	34
5	Wolfe	-3	3.748	-4	4.248	36	19.858	101.688	19.866	36
6	Wolfe	5	3.748	3	4.248	23	19.858	30.333	19.866	35

Исходя из экспериментальных данных конечное решение в каждом из экспериментов сходится к одному, почти равному истинному. Но быстрее всего мы пришли к решению в четвертом эксперименте.

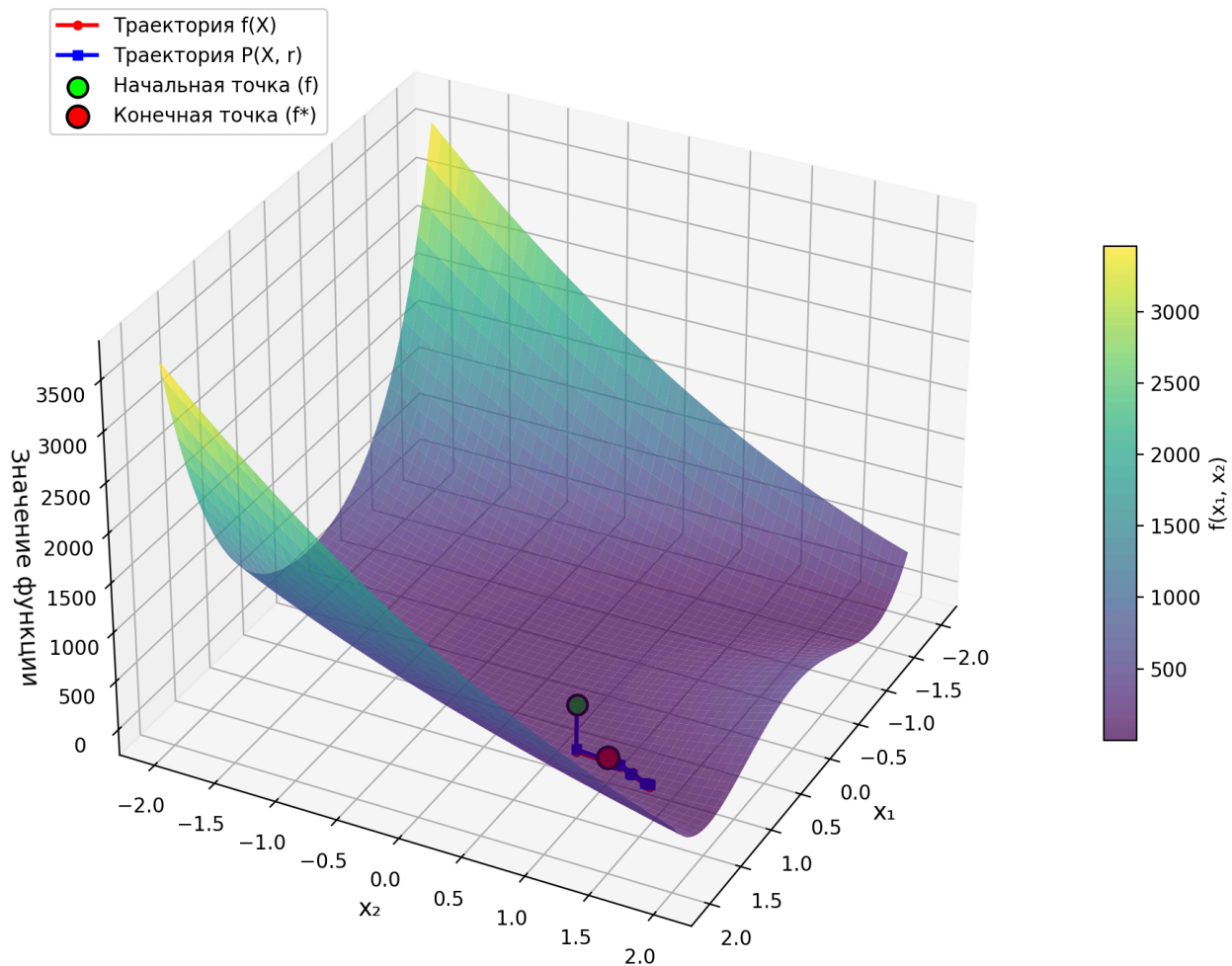
3.8 Проверка решения на функции Розенброка с линейным и кубическим ограничениями.



Для поиска решения будем выбирать шаг стратегией Вульфа, показал быстрее в предыдущем эксперименте.

Начнем с точек $x_1 = 0.5, x_2 = 0.5$, гессиан и градиент будем искать численными методами, так как сложно дифференцировать исходную функцию.

Поверхность Розенброка $f(x_1, x_2)$ с наложенными траекториями f и $P(X, r)$



Начальная точка: (0.5, 0.5)

$f = 6.5$

$h1 = x2 - (x1^3 - 1) = 0.375$

$g1 = 5 - x1 - x2 = 3.0$

$P(X, r) = 49.87780286292946$

$r = 10$

iter 0: $X=(1.0514, 0.7733)$, $||grad||=9.08e+01$

Сошлись за 5 итераций

$X = (1.196144, 1.429639)$

$f = 0.038598$

$h1 = -2.82e-01$

$g1 = 1.37e+00$

$P(X, r) = 22.69554389440788$

```

r = 2.5
iter 0: X=(1.1756, 1.3837), ||grad||=2.09e+00
Сошлись за 3 итераций
X = (1.174108, 1.380609)
f = 0.030746
h1 = -2.38e-01
g1 = 1.45e+00
P(X, r) = 5.736386958992484

```

```

r = 0.625
iter 0: X=(1.1183, 1.2500), ||grad||=1.72e+00
Сошлись за 4 итераций
X = (1.107860, 1.229931)
f = 0.012298
h1 = -1.30e-01
g1 = 1.66e+00
P(X, r) = 1.4819259096032817

```

```

r = 0.15625
iter 0: X=(1.0516, 1.1043), ||grad||=1.43e+00
Сошлись за 4 итераций
X = (1.036286, 1.075296)
f = 0.001515
h1 = -3.76e-02
g1 = 1.89e+00
P(X, r) = 0.383912528572234

```

```

r = 0.0390625
iter 0: X=(1.0110, 1.0220), ||grad||=6.99e-01
Сошлись за 4 итераций
X = (1.007153, 1.014839)
f = 0.000074
h1 = -6.77e-03
g1 = 1.98e+00
P(X, r) = 0.0973313456960712

```

```
r = 0.009765625
iter 0: X=(1.0013, 1.0027), ||grad||=2.35e-01
Сошлись за 3 итераций
X = (1.001099, 1.002333)
f = 0.000003
h1 = -9.67e-04
g1 = 2.00e+00
P(X, r) = 0.02440147747564109
```

```
r = 0.00244140625
iter 0: X=(1.0002, 1.0004), ||grad||=6.58e-02
Сошлись за 2 итераций
X = (1.000162, 1.000360)
f = 0.000000
h1 = -1.27e-04
g1 = 2.00e+00
P(X, r) = 0.006103037496589016
```

```
r = 0.0006103515625
iter 0: X=(1.0000, 1.0001), ||grad||=1.73e-02
Сошлись за 2 итераций
X = (1.000025, 1.000059)
f = 0.000000
h1 = -1.63e-05
g1 = 2.00e+00
P(X, r) = 0.001525859540047649
```

```
r = 0.000152587890625
iter 0: X=(1.0000, 1.0000), ||grad||=4.42e-03
Сошлись за 2 итераций
X = (1.000004, 1.000011)
f = 0.000000
h1 = -2.07e-06
g1 = 2.00e+00
```


$P(X, r) = 0.00038146885774269847$

$r = 3.814697265625e-05$

iter 0: $X=(1.0000, 1.0000)$, $||grad||=1.12e-03$

Сошлись за 1 итераций

$X = (1.0000001, 1.0000002)$

$f = 0.0000000$

$h1 = -2.60e-07$

$g1 = 2.00e+00$

$P(X, r) = 9.536738821391864e-05$

$r = 9.5367431640625e-06$

iter 0: $X=(1.0000, 1.0000)$, $||grad||=2.81e-04$

Сошлись за 1 итераций

$X = (1.0000000, 1.0000000)$

$f = 0.0000000$

$h1 = -3.26e-08$

$g1 = 2.00e+00$

$P(X, r) = 2.3841855538965062e-05$

$r = 2.384185791015625e-06$

iter 0: $X=(1.0000, 1.0000)$, $||grad||=7.05e-05$

Сошлись за 1 итераций

$X = (1.0000000, 1.0000000)$

$f = 0.0000000$

$h1 = -4.08e-09$

$g1 = 2.00e+00$

$P(X, r) = 5.960464340107112e-06$

$r = 5.960464477539062e-07$

iter 0: $X=(1.0000, 1.0000)$, $||grad||=1.76e-05$

Сошлись за 1 итераций

$X = (1.0000000, 1.0000000)$

$f = 0.0000000$

$h1 = -5.10e-10$

$$g1 = 2.00e+00$$

$$P(X, r) = 1.490116111132528e-06$$

Итог

$$X^* = (1.000000, 1.000000)$$

$$f^* = 0.000000$$

$$\text{Проверка } h1 = x2 - (x1^3 - 1) = -5.10e-10$$

$$\text{Проверка } g1 = 5 - x1 - x2 = 2.00e+00$$

46 кол-во итераций

4. Вывод

В ходе работы мы с помощью метода штрафных функций и метода Ньютона оптимизировали функцию с ограничениями. Мы аппроксимировали исходную функцию и получили новую, схожую с исходной, но не имеющую ограничений. Провели несколько экспериментов с исходной функцией и с функцией Розенброка.

Приложения

1. Код программы графического представления функции и ограничений

```
import numpy as np
import matplotlib.pyplot as plt

def objective(x):
    return x[0]**2 + x[1]**2 - x[0] - 2*x[1]

x1 = np.linspace(-2, 12, 400)
x2 = np.linspace(-2, 12, 400)
X1, X2 = np.meshgrid(x1, x2)
Z = objective([X1, X2])

plt.figure(figsize=(12, 9))

contour = plt.contour(X1, X2, Z, levels=30, cmap='viridis',
alpha=0.7)
plt.colorbar(contour, label='Значение целевой функции
f(x)')

x_constraint1 = np.linspace(-2, 12, 100)
y_constraint1 = (16 - x_constraint1) / 2
plt.plot(x_constraint1, y_constraint1, 'b-', linewidth=2,
label='x1 + 2*x2 = 16')

x_fill1 = np.linspace(-2, 12, 100)
y_fill1 = (16 - x_fill1) / 2
```

```

plt.fill_between(x_fill, -2, y_fill, alpha=0.2,
color='blue', label='x1 + 2*x2 ≤ 16')

x_eq = np.linspace(-2, 12, 100)
y_eq = 8 - x_eq
plt.plot(x_eq, y_eq, 'r-', linewidth=2, label='x1 + x2 = 8')

plt.axvline(x=0, color='gray', linestyle='--', alpha=0.5,
label='x1 = 0')
plt.axhline(y=0, color='gray', linestyle='--', alpha=0.5,
label='x2 = 0')

corner_points = [(0, 8), (8, 0)]
for point in corner_points:
    plt.scatter(point[0], point[1], color='orange', s=100,
marker='o', zorder=3)

x_feasible = np.linspace(0, 8, 100)
y_feasible = 8 - x_feasible
plt.plot(x_feasible, y_feasible, 'g-', linewidth=3,
alpha=0.5, label='Допустимый отрезок')

plt.xlabel('x1', fontsize=12)
plt.ylabel('x2', fontsize=12)
plt.title('Графическое представление задачи
ОПТИМИЗАЦИИ\nmin f(x) = x12 + x22 - x1 - 2x2', fontsize=14)
plt.grid(True, alpha=0.3)
plt.legend(loc='best')
plt.axis('equal')

```

```
plt.xlim(-1, 10)
plt.ylim(-1, 10)

plt.tight_layout()
plt.show()
```

2. Код программы решения задачи с помощью заданных методов

```
import numpy as np

import matplotlib.pyplot as plt


def f(X):

    x1, x2 = X

    return x1 ** 2 + x2 ** 2 - x1 - 2 * x2


def h1(X):

    x1, x2 = X

    return x1 + x2 - 8


def g1(X):

    x1, x2 = X

    return 16 - x1 - 2 * x2


def g2(X):

    x1, _ = X

    return x1


def g3(X):

    _, x2 = X

    return x2


def P(X, r):

    x1, x2 = X

    f_val = x1 ** 2 + x2 ** 2 - x1 - 2 * x2
```

```

    h_val = h1(X)

    g1_val = g1(X)

    g2_val = g2(X)

    g3_val = g3(X)

    penalty_eq = (1.0 / np.sqrt(r)) * h_val ** 2

    penalty_ineq = r * (1.0 / g1_val + 1.0 / g2_val + 1.0 /
g3_val)

    return f_val + penalty_eq + penalty_ineq

def gradient_P(X, r):
    x1, x2 = X

    h = x1 + x2 - 8

    g1_val = 16 - x1 - 2 * x2

    g2_val = x1

    g3_val = x2

    df_dx1 = 2 * x1 - 1

    df_dx2 = 2 * x2 - 2

    factor_eq = 2.0 / np.sqrt(r)

    d_eq_dx1 = factor_eq * h

    d_eq_dx2 = factor_eq * h

    d_ineq_dx1 = r * (1.0 / g1_val ** 2 - 1.0 / g2_val ** 2)

    d_ineq_dx2 = r * (2.0 / g1_val ** 2 - 1.0 / g3_val ** 2)

```



```

        return np.array([df_dx1 + d_eq_dx1 + d_ineq_dx1,
                          df_dx2 + d_eq_dx2 + d_ineq_dx2])

def hessian_P(X, r):
    x1, x2 = X

    g1_val = 16 - x1 - 2 * x2
    g2_val = x1
    g3_val = x2

    factor_eq = 2.0 / np.sqrt(r)

    H11 = 2 + factor_eq + r * (2.0 / g1_val ** 3 + 2.0 /
g2_val ** 3)
    H22 = 2 + factor_eq + r * (8.0 / g1_val ** 3 + 2.0 /
g3_val ** 3)
    H12 = factor_eq + r * (4.0 / g1_val ** 3)

    return np.array([[H11, H12], [H12, H22]])

def newton_step(X, r):
    grad = gradient_P(X, r)
    H = hessian_P(X, r)

    return np.linalg.solve(H, -grad)

X = np.array([-3, -4])
r = 10

```

```

history_X = []

history_f = []

history_P = []

print(f"Начальная точка: ({X[0]}, {X[1]})")

print(f"f = {f(X)}")

print(f"h1 = x1 + x2 - 8 = {h1(X)}")

print(f"g1 = 16 - x1 - 2x2 = {g1(X)}")

print(f"P(X, r) = {P(X, r)}")

history_X.append(X.copy())

history_f.append(f(X))

history_P.append(P(X, r))

a = 0

iter_counter = 0

# Поиск шага Вульфа

def line_search_wolfe(X, delta, r, grad, c1=1e-4, c2=0.9,
alpha_max=1.0, rho=0.5, max_iter=100):

    alpha = alpha_max

    P_current = P(X, r)

    grad_current = grad

    for _ in range(max_iter):

        X_new = X + alpha * delta

        if P(X_new, r) > P_current + c1 * alpha *
grad_current @ delta:

```

```

        alpha *= rho

        continue

    grad_new = gradient_P(X_new, r)
    if grad_new @ delta < c2 * grad_current @ delta:
        if alpha < alpha_max:
            alpha = min(alpha_max, alpha * 1.5)
            continue

    return alpha

return alpha * 0.1

for outer_iter in range(20):
    print(f"\nr = {r}")

    for inner_iter in range(50):
        grad = gradient_P(X, r)
        grad_norm = np.linalg.norm(grad)
        a += 1
        iter_counter += 1

        if grad_norm < 1e-6:
            print(f"    Сошлись за {inner_iter} итераций")
            break

    delta = newton_step(X, r)

```

```

        alpha = line_search_wolfe(X, delta, r, grad)

        # Поиск шага Armijo
        # alpha = 1.0
        # P_current = P(X, r)
        # while alpha > 1e-8:
        #     X_new = X + alpha * delta
        #     if P(X_new, r) < P_current + 0.5 * alpha *
grad @ delta:
        #         break
        #     alpha *= 0.5

    X = X + alpha * delta

    history_X.append(X.copy())
    history_f.append(f(X))
    history_P.append(P(X, r))

    if inner_iter % 10 == 0:
        print(f"  iter {inner_iter:2d}: X=({X[0]:.4f},
{X[1]:.4f}), ||grad||={grad_norm:.2e}")

    print(f"  X = ({X[0]:.6f}, {X[1]:.6f})")
    print(f"  f = {f(X):.6f}")
    print(f"  h1 = {h1(X):.2e}")
    print(f"  g1 = {g1(X):.2e}")
    print(f"P(X, r) = {P(X, r)}")

```

```

    if r < 1e-6 or abs(P(X, r) - f(X)) < 1e-6:
        break

    r = r / 4 # уменьшаем штраф

print("\nИтог")
print(f"x* = ({X[0]:.6f}, {X[1]:.6f})")
print(f"f* = {f(X):.6f}")
print(f"Проверка x1 + x2 - 8 = {h1(X):.2e}")
print(f"Теоретический оптимум: (3.75, 4.25), f = 19.875")

print(a, "кол-во итераций")

# ГРАФИК: одна поверхность f, на ней траектории для f и P
x1_grid = np.linspace(0, 6, 100)
x2_grid = np.linspace(0, 6, 100)
X1, X2 = np.meshgrid(x1_grid, x2_grid)

Z_f = X1**2 + X2**2 - X1 - 2*X2

history_X = np.array(history_X)
Z_f_traj = np.array(history_f)
Z_P_traj = np.array(history_P)

fig = plt.figure(figsize=(14, 9))
ax = fig.add_subplot(111, projection='3d')

```

```

# Поверхность  $f(x_1, x_2)$ 
surf = ax.plot_surface(X1, X2, Z_f, cmap='viridis',
alpha=0.7, edgecolor='none')

# Траектория значений  $f$  (красная)
ax.plot(history_X[:, 0], history_X[:, 1], Z_f_traj,
        'r-o', linewidth=2, markersize=4, label='Траектория  
 $f(x)$ ')

# Траектория значений  $P$  (синяя)
ax.plot(history_X[:, 0], history_X[:, 1], Z_P_traj,
        'b-s', linewidth=2, markersize=4, label='Траектория  
 $P(X, r)$ ')

# Начальная точка (зеленая)
ax.scatter(history_X[0, 0], history_X[0, 1], Z_f_traj[0],
          color='lime', s=100, edgecolor='black',
linewidth=1.5, label='Начальная точка ( $f$ )')

# Конечная точка (красная)
ax.scatter(history_X[-1, 0], history_X[-1, 1], Z_f_traj[-1],
          color='red', s=120, edgecolor='black',
linewidth=1.5, label='Конечная точка ( $f^*$ )')

ax.set_xlabel('x1', fontsize=12)
ax.set_ylabel('x2', fontsize=12)
ax.set_zlabel('Значение функции', fontsize=12)

```

```

ax.set_title('Поверхность  $f(x_1, x_2)$  с наложенными
траекториями  $f$  и  $P(X, r)$ ', fontsize=14)
ax.legend(fontsize=10, loc='upper left')

# Цветовая шкала для поверхности  $f$ 
fig.colorbar(surf, ax=ax, shrink=0.5, aspect=15,
label='f(x1, x2)')

plt.tight_layout()
plt.show()

```

3. Код программы решения задачи с исходной функцией Розенброка

```
import numpy as np
import matplotlib.pyplot as plt

def f(X):
    x1, x2 = X
    return (1 - x1) ** 2 + 100 * (x2 - x1 ** 2) ** 2

def h1(X):
    x1, x2 = X
    return x2 - (x1 ** 3)

def g1(X):
    x1, x2 = X
    return 4 - x1 - x2

def g2(X):
    x1, _ = X
    return x1

def g3(X):
    _, x2 = X
```



```

    return x2

def P(X, r):
    x1, x2 = X
    f_val = f(X)
    h_val = h1(X)
    g1_val = g1(X)
    g2_val = g2(X)
    g3_val = g3(X)

    penalty_eq = (1.0 / np.sqrt(r)) * h_val ** 2
    penalty_ineq = r * (1.0 / g1_val + 1.0 / g2_val + 1.0 /
g3_val)

    return f_val + penalty_eq + penalty_ineq

def gradient_P(X, r, eps=1e-8):
    """Численный градиент"""
    grad = np.zeros_like(X)
    for i in range(len(X)):
        X_plus = X.copy()
        X_minus = X.copy()
        X_plus[i] += eps
        X_minus[i] -= eps
        grad[i] = (P(X_plus, r) - P(X_minus, r)) / (2 * eps)

```

```

return grad

def hessian_P(X, r, eps=1e-6):
    """Численный гессиан"""
    n = len(X)
    H = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            X_plus_i = X.copy()
            X_minus_i = X.copy()
            X_plus_i[i] += eps
            X_minus_i[i] -= eps

            if i == j:
                # Вторая производная по одной переменной
                H[i, j] = (P(X_plus_i, r) - 2 * P(X, r) +
P(X_minus_i, r)) / (eps ** 2)
            else:
                # Смешанная производная
                X_plus_j = X.copy()
                X_minus_j = X.copy()
                X_plus_j[j] += eps
                X_minus_j[j] -= eps

                X_plus_ij = X.copy()
                X_plus_ij[i] += eps

```

```

        X_plus_ij[j] += eps

        X_minus_ij = X.copy()

        X_minus_ij[i] -= eps

        X_minus_ij[j] -= eps

        H[i, j] = (P(X_plus_ij, r) - P(X_plus_i, r)
- P(X_plus_j, r) + P(X, r) +
                    P(X_minus_ij, r) - P(X_minus_i,
r) - P(X_minus_j, r) + P(X, r)) / (2 * eps ** 2)

    return H

def newton_step(X, r):
    grad = gradient_P(X, r)
    H = hessian_P(X, r)
    try:
        return np.linalg.solve(H, -grad)
    except np.linalg.LinAlgError:
        return -grad # fallback на градиентный спуск

X = np.array([0.5, 0.5])
r = 10

history_X = []
history_f = []
history_P = []

```

```

print(f"Начальная точка: ({X[0]}, {X[1]})")

print(f"f = {f(X)}")

print(f"h1 = x2 - (x1^3 - 1) = {h1(X)}")

print(f"g1 = 5 - x1 - x2 = {g1(X)}")

print(f"P(X, r) = {P(X, r)}")

history_X.append(X.copy())

history_f.append(f(X))

history_P.append(P(X, r))

a = 0

iter_counter = 0

# Поиск шага Вульфа

def line_search_wolfe(X, delta, r, grad, c1=1e-4, c2=0.9,
alpha_max=1.0, rho=0.5, max_iter=100):

    alpha = alpha_max

    P_current = P(X, r)

    grad_current = grad

    for _ in range(max_iter):

        X_new = X + alpha * delta

        if P(X_new, r) > P_current + c1 * alpha *
grad_current @ delta:

            alpha *= rho

            continue

```

```

        grad_new = gradient_P(X_new, r)
        if grad_new @ delta < c2 * grad_current @ delta:
            if alpha < alpha_max:
                alpha = min(alpha_max, alpha * 1.5)
                continue

        return alpha

    return alpha * 0.1

for outer_iter in range(20):
    print(f"\nr = {r}")

    for inner_iter in range(50):
        grad = gradient_P(X, r)
        grad_norm = np.linalg.norm(grad)
        a += 1
        iter_counter += 1

        if grad_norm < 1e-6:
            print(f"    Сошлись за {inner_iter} итераций")
            break

    delta = newton_step(X, r)

```

```

        alpha = line_search_wolfe(X, delta, r, grad)

        X = X + alpha * delta

        history_X.append(X.copy())
        history_f.append(f(X))
        history_P.append(P(X, r))

        if inner_iter % 10 == 0:
            print(f"    iter {inner_iter:2d}: X=({X[0]:.4f},
{X[1]:.4f}), ||grad||={grad_norm:.2e}")

        print(f"    X = ({X[0]:.6f}, {X[1]:.6f})")
        print(f"    f = {f(X):.6f}")
        print(f"    h1 = {h1(X):.2e}")
        print(f"    g1 = {g1(X):.2e}")
        print(f"P(X, r) = {P(X, r)}")

        if r < 1e-6 or abs(P(X, r) - f(X)) < 1e-6:
            break

        r = r / 4

print("\nИТОГ")
print(f"X* = ({X[0]:.6f}, {X[1]:.6f})")
print(f"f* = {f(X):.6f}")
print(f"Проверка h1 = x2 - (x1^3 - 1) = {h1(X):.2e}")

```

```

print(f"Проверка  $g1 = 5 - x1 - x2 = \{g1(X):.2e\}$ ")

print(a, "кол-во итераций")

# ГРАФИК: поверхность f, на ней траектории для f и P
x1_grid = np.linspace(-2, 2, 100)
x2_grid = np.linspace(-2, 2, 100)
X1, X2 = np.meshgrid(x1_grid, x2_grid)

Z_f = np.zeros_like(X1)
for i in range(X1.shape[0]):
    for j in range(X1.shape[1]):
        Z_f[i, j] = (1 - X1[i, j]) ** 2 + 100 * (X2[i, j] -
X1[i, j] ** 2) ** 2

history_X = np.array(history_X)
Z_f_traj = np.array(history_f)
Z_P_traj = np.array(history_P)

fig = plt.figure(figsize=(14, 9))
ax = fig.add_subplot(111, projection='3d')

# Поверхность f(x1, x2)
surf = ax.plot_surface(X1, X2, Z_f, cmap='viridis',
alpha=0.7, edgecolor='none')

# Траектория значений f (красная)
ax.plot(history_X[:, 0], history_X[:, 1], Z_f_traj,

```

```

        'r-o', linewidth=2, markersize=4, label='Траектория
f(x)')

# Траектория значений P (синяя)
ax.plot(history_X[:, 0], history_X[:, 1], z_P_traj,
        'b-s', linewidth=2, markersize=4, label='Траектория
P(X, r)')

ax.scatter(history_X[0, 0], history_X[0, 1], z_f_traj[0],
          color='lime', s=100, edgecolor='black',
linewidth=1.5, label='Начальная точка (f)')

ax.scatter(history_X[-1, 0], history_X[-1, 1], z_f_traj[-1],
          color='red', s=120, edgecolor='black',
linewidth=1.5, label='Конечная точка (f*)')

ax.set_xlabel('x1', fontsize=12)
ax.set_ylabel('x2', fontsize=12)
ax.set_zlabel('Значение функции', fontsize=12)
ax.set_title('Поверхность Розенброка f(x1, x2) с
наложенными траекториями f и P(X, r)', fontsize=14)
ax.legend(fontsize=10, loc='upper left')

fig.colorbar(surf, ax=ax, shrink=0.5, aspect=15,
label='f(x1, x2)')

plt.tight_layout()
plt.show()

```